FINAL CAPSTONE PROJECT PRESENTATION

DevOps Incident & Task Tracker

Production DevSecOps Reference Architecture & Implementation

AWS EKS • Terraform IaC • GitHub Actions CI/CD • Aqua Trivy • ArgoCD GitOps • Prometheus & Grafana

Repository: https://github.com/BYT-Bender/devsecops-python

EXECUTIVE SUMMARY

Project Overview & Core Objectives

📌 Problem Statement & Vision

- **Cloud-Native Complexity:** Modern delivery requires high deployment velocity without sacrificing security guardrails, infrastructure auditability, or runtime visibility.
- **Vulnerability Gaps:** In typical legacy setups, security scanning is manual, ad-hoc, or non-blocking, leading to production CVE exposure.
- **Configuration Drift:** Direct cluster updates without Git version control produce fragile, unreproducible environments.
- **Engineered Solution:** A reference DevSecOps implementation establishing shift-left container security, declarative GitOps, and end-to-end metrics.

🎯 Implementation Milestones

- **Full-Stack Application:** Containerized FastAPI backend, PostgreSQL database with Alembic migrations, and modern React 18 UI.
- **Automated Testing:** Pytest test suite covering 10 test cases across health, readiness, validation, and full CRUD flows.
- **DevSecOps Hard Gate:** CI pipeline fails on HIGH or CRITICAL unpatched container CVEs via Trivy.
- **Cloud Infrastructure:** Multi-AZ AWS VPC and EKS v1.31 in ap-south-1 managed entirely via Terraform.
- **GitOps & Observability:** ArgoCD automated reconciliation; Prometheus target discovery and Grafana KPIs.

SYSTEM ARCHITECTURE

End-to-End DevSecOps Architecture Flow

1. Continuous Integration (CI)

- **Developer Push:** Code pushed to GitHub triggers GitHub Actions workflow.
- **Stage 1 (Test):** Python 3.12 executes 10 Pytest cases; Node 22 builds frontend bundle.
- **Stage 2 (Build & Scan):** Builds `linux/amd64` images tagged with immutable Git SHA.
- **Trivy Security Gate:** Scans both images with `--exit-code 1` on HIGH/CRITICAL CVEs.
- **Registry Push:** Verified artifacts published to GitHub Container Registry (`ghcr.io`).

2. Declarative GitOps

- **Automated GitOps Commit:** Pinned `yq v4.44.3` updates image SHA tags in `values-prod.yaml` with `[skip ci]` .
- **Zero Cluster Access in CI:** CI pipeline has zero Kubernetes credentials, preventing credential exposure.
- **ArgoCD Controller:** In-cluster controller watches Git repo, detects new commit, and synchronizes.
- **Self-Healing:** Automatically remediates manual drift and prunes removed resources.

3. Cloud Runtime & Observability

- **AWS EKS (ap-south-1):** VPC `10.20.0.0/16` , public/private subnets, NAT gateway, managed node groups.
- **Ingress Routing:** NGINX Ingress Controller backed by AWS NLB routing `/` and `/api` .
- **Workloads:** 2× Backend (non-root UID 10001), 2× Frontend (UID 101), PostgreSQL with PVC.
- **Monitoring:** Prometheus scrapes `/metrics` via ServiceMonitor; Grafana provides live KPIs.

TOOLS & TECHNOLOGIES

Multi-Layered Technology Stack

Backend & Database

- **FastAPI 0.115:** High-performance async REST API
- **SQLAlchemy 2 & Alembic:** ORM & DB migrations
- **PostgreSQL 16:** Production persistent relational DB

Frontend Layer

- **React 18 + Vite 6:** Lightweight, reactive SPA UI
- **Nginx Unprivileged:** Alpine runtime on port 8080
- **Non-Root User:** Runs as UID 101 for security

CI/CD & DevSecOps

- **GitHub Actions:** Automated 2-stage pipeline
- **Aqua Trivy:** Container CVE scanning hard gate
- **GHCR:** Immutable Git commit SHA tagging

Infrastructure as Code

- **Terraform ≥ 1.7:** Declarative AWS provisioning
- **AWS VPC:** Multi-AZ subnets + NAT Gateway
- **AWS EKS 1.31:** Managed Kubernetes cluster

Kubernetes & GitOps

- **Helm 3:** Parametrized application packaging
- **NGINX Ingress:** Integrated with AWS NLB
- **ArgoCD:** Automated pull-based GitOps sync

Full Observability

- **Prometheus Operator:** Automated scraping
- **ServiceMonitor CRD:** Discovers backend pods
- **Grafana Dashboard:** Auto-provisioned sidecar

CONTINUOUS INTEGRATION

Two-Stage GitHub Actions CI/CD Pipeline

Stage 1: Test & Code Quality (All Events)

- **Execution Trigger:** Runs on all Pull Requests, pushes, and manual workflow dispatches.
- **Isolated Environment:** Ubuntu latest runner with Python 3.12 and Node.js 22.
- **Pytest Execution:** Runs `pytest -v` across 10 unit and API tests with SQLite isolation.
- **Complete Coverage Gate:** Tests health, readiness, CRUD endpoints, and 422 input validation.
- **Frontend Build Verification:** `npm ci && npm run build` verifies asset bundling.

Stage 2: Build, Scan & GHCR Push (Main Branch)

- **Condition:** Executes only upon push to `main` after Stage 1 passes 100%.
- **Name Normalization:** Lowercases repo owner to prevent GHCR uppercase registry pull failures.
- **Architecture Targeting:** Explicitly builds for `linux/amd64` to guarantee EKS node compatibility.
- **Trivy Hard Gate:** Scans both images; halts build (`exit-code 1`) on HIGH/CRITICAL CVEs.
- **GitOps Update:** Pinned `yq v4.44.3` updates `values-prod.yaml` and pushes with `[skip ci]` .

SECURITY SCANNING (M6)

DevSecOps: Shift-Left Security & Trivy Vulnerability Gate

Trivy Security Hard Gate Policy

- **Shift-Left Architecture:** Security inspection occurs in CI prior to any image being pushed to GHCR.
- **Severity Filter:** Targets `--severity HIGH,CRITICAL` vulnerabilities.
- **Enforced Exit Code:** Configured with `--exit-code 1` — any unpatched flaw breaks the build.
- **Noise Reduction:** Uses `--ignore-unfixed` to focus strictly on actionable CVEs with vendor patches.
- **Comprehensive Coverage:** Dual scan of both FastAPI backend and Nginx frontend images.

Real-World CVE Remediation Case Study

- **Identified CVEs:** Initial scan caught 3 HIGH vulnerabilities in Starlette 0.41.3:
  - **CVE-2025-62727:** HTTP Range header DoS in FileResponse
  - **CVE-2026-48818:** Windows StaticFiles path SSRF / NTLM leak
  - **CVE-2026-54283:** URL-encoded form parsing DoS
- **Root Cause:** Older `prometheus-fastapi-instrumentator 7.0.2` forced Starlette <1.0.0.
- **Resolution:** Upgraded Instrumentator to 8.1.0 and Starlette to 1.3.1; Trivy scan passed with zero findings!

INFRASTRUCTURE AS CODE (M7)

AWS Infrastructure as Code with Terraform

AWS Cloud Architecture (ap-south-1)

- **VPC CIDR 10.20.0.0/16:** Provisioned across 2 availability zones (`ap-south-1a` and `ap-south-1b`).
- **Subnet Segmentation:** 2 public subnets for external load balancers; 2 private subnets for EKS compute nodes.
- **NAT Gateway:** Managed NAT Gateway enables secure outbound internet access for private pods.
- **EKS Cluster v1.31:** Managed Kubernetes control plane with public API endpoint and IAM OIDC provider.
- **Managed Node Group:** Autoscaling EC2 `t3.medium` instances (2 to 4 worker nodes).

Operational Governance & Security

- **Zero Secrets in Git:** Enforces `terraform.tfvars.example` ; sensitive files ignored via `.gitignore` .
- **Resource Tagging:** Enforces tags: `Environment=dev` , `Project=devsecops-capstone` , `ManagedBy=terraform` .
- **Execution Flow:** Deterministic `terraform init` → `terraform plan` → `terraform apply` .
- **Clean Lifecycle Teardown:** Verified `terraform destroy` prevents runaway AWS cloud billing.

KUBERNETES & HELM (M8)

Container Hardening & Helm Packaging

Multi-Stage Non-Root Containers

- **Backend Container:** Built on `python:3.12-slim` with non-root user `appuser` (UID 10001). Pre-runs Alembic migrations before Uvicorn.
- **Frontend Container:** 2-stage build: compiles React in Node 22, then serves via `nginxinc/nginx-unprivileged:alpine`.
- **Non-Root Runtime:** Frontend executes as UID 101 on non-privileged port 8080.
- **Local Development:** Single command `docker compose up --build` launches the complete stack locally.

Helm Chart Architecture (helm/taskboard)

- **Parametrized Packaging:** Unified chart for Backend, Frontend, PostgreSQL, Ingress, HPA, and ServiceMonitor.
- **Environment Profiles:** `values.yaml` for dev; `values-prod.yaml` for prod (2 replicas, SHA tags, Ingress enabled).
- **Ingress Routing:** AWS NLB terminates traffic, routing `/` to Frontend (port 80) and `/api` to Backend (port 8000).
- **High Availability:** 2 replicas per microservice with liveness and readiness probes.

GITOPS CONTINUOUS DELIVERY

Declarative GitOps Delivery with ArgoCD

GitOps Principles & Zero-Access CI

- **Git as Single Source of Truth:** Cluster state is declared in version-controlled Git (`helm/taskboard`).
- **Pull-Based Delivery:** ArgoCD controller inside EKS pulls changes from Git, rather than CI pushing into the cluster.
- **Attack Surface Reduction:** GitHub Actions requires zero AWS/K8s credentials, preventing credential exposure.
- **Auditability & Traceability:** Every deployment maps to an immutable Git commit SHA.

Reconciliation & Self-Healing Controller

- **Application CRD:** Configured via `k8s/argocd-application.yaml` targeting the `taskboard` namespace.
- **Automated Sync:** Instantly deploys new image SHA tags committed by CI to `values-prod.yaml`.
- **Self-Healing (selfHeal: true):** Automatically detects and reverts manual out-of-band `kubectl` modifications.
- **Resource Pruning (prune: true):** Cleans up obsolete resources automatically upon deletion in Git.

OBSERVABILITY (M9)

Full-Stack Observability: Prometheus & Grafana

Prometheus Metrics Collection Pipeline

- **Native Instrumentation:** FastAPI routes instrumented with Prometheus Instrumentator at `/metrics`.
- **Metrics Tracked:** HTTP request volume, response latency histograms, error counters, and task entities.
- **kube-prometheus-stack:** Deployed via Helm into the `monitoring` namespace.
- **ServiceMonitor CRD:** Declaratively directs Prometheus to scrape backend pods on port 8000 every 15s.
- **Target Verification:** Prometheus Targets UI confirms backend target status as `UP`.

Auto-Provisioned Grafana Dashboard

- **Sidecar ConfigMap:** Grafana sidecar automatically imports dashboards labeled `grafana_dashboard: "1"`.
- **Panel 1 (RPS):** Real-time Request Rate by API handler and status code.
- **Panel 2 (Latency):** 95th percentile request latency curves under load.
- **Panel 3 (Errors):** Real-time detection of 4xx client and 5xx server errors.
- **Panel 4 (Tasks):** Application business metric displaying total tasks handled.
- **Load Testing:** Validated live using `scripts/load-test.sh` (500 requests).

KEY LEARNINGS

Engineering Challenges & Production Lessons

Challenges Overcome

- **Transitive Dependency Conflicts:** Starlette CVEs could not be patched until identifying that the older Prometheus instrumentator pinned Starlette to <1.0.0.
- **GHCR Case-Sensitivity:** Uppercase characters in GitHub repository owner names caused registry push failures; resolved using bash string transformation.
- **Ingress NLB Annotations:** Configured AWS Network Load Balancer annotations in NGINX Ingress to ensure clean layer-4 pass-through.
- **CRD Dependency Ordering:** ServiceMonitor requires Prometheus Operator CRDs to exist prior to installation.

Core Production Takeaways

- **Immutable Git SHAs > :latest:** Eliminates image ambiguity, guarantees deterministic deployments, and makes rollbacks instantaneous.
- **Shift-Left Security:** Enforcing automated vulnerability gates in CI blocks CVEs before they ever reach registries or clusters.
- **GitOps Eliminates Human Error:** Removing cluster write permissions from CI and developers ensures all changes are peer-reviewed in Git.
- **Cloud Cost Hygiene:** Clean IaC teardown scripts (`terraform destroy`) prevent unexpected cloud bills.

DEMO ROADMAP

Live Project Demonstration & Rubric Verification

Rubric Verification Checklist (M1 – M10)

- **M1 – M4 (App, Tests, Docker):** FastAPI + React running locally via Docker Compose; 10 Pytest tests passing; multi-stage non-root containers.
- **M5 – M6 (CI/CD & DevSecOps):** GitHub Actions 2-stage pipeline green; Trivy scan gate passed with 0 HIGH/CRITICAL CVEs; published to GHCR.
- **M7 (AWS Terraform IaC):** VPC (10.20.0.0/16) and EKS v1.31 planned and provisioned in ap-south-1.
- **M8 (Kubernetes & Helm):** Deployments running 2 replicas each, Ingress routing healthy.
- **M9 (Observability):** Prometheus targets UP; live Grafana dashboard graphing RPS, latency, and errors.
- **M10 (Documentation & Video):** Technical README.md, PPT slides, and walkthrough video committed to demo/.

Live Walkthrough Agenda

- **1. Codebase Overview:** Folder structure, models, schemas, Dockerfiles, Helm templates, Terraform.
- **2. CI/CD Live Run:** Push to GitHub, observe Pytest, Trivy scan, GHCR push, and GitOps commit.
- **3. Terraform IaC:** Review terraform plan and AWS Console proof of VPC and EKS.
- **4. ArgoCD GitOps:** Show ArgoCD application synchronization and self-healing.
- **5. Kubernetes Cluster Health:** Inspect pods, services, and ingress on EKS.
- **6. Observability in Action:** Prometheus Targets page and live traffic generation on Grafana dashboard.